Q1. Multiple Threads Using a Single Class

Score: 10.00 **TC: 3/3 passed**

CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int number = sc.nextInt();

        MyThread t1 = new MyThread("Prime", number);
        MyThread t2 = new MyThread("Armstrong", number);
        MyThread t3 = new MyThread("Reverse", number);

        t1.start();
        try {
            t1.join();
        } catch (InterruptedException e) {}

        t2.start();
        try {
            t2.join();
        } catch (InterruptedException e) {}

        t3.start();
        try {
            t3.join();
        } catch (InterruptedException e) {}
    }
}

class MyThread extends Thread {
    int number;

    MyThread(String name, int number) {
        super(name);
        this.number = number;
    }

    public void run() {
        String threadName = getName();

        if (threadName.equals("Prime")) {
            boolean isPrime = true;
            if (number < 2) {
                isPrime = false;
            } else {
                for (int i = 2; i <= Math.sqrt(number); i++) {
                    if (number % i == 0) {
                        isPrime = false;
                        break;
                    }
                }
            }
            if (isPrime) {
                System.out.println("Prime Thread: " + number + " is a prime number");
            } else {
                System.out.println("Prime Thread: " + number + " is not a prime number");
            }

        } else if (threadName.equals("Armstrong")) {
            int temp = number, sum = 0;
            int digits = String.valueOf(number).length();
```

```
            while (temp > 0) {
                int digit = temp % 10;
                sum += Math.pow(digit, digits);
                temp /= 10;
            }
            if (sum == number) {
                System.out.println("Armstrong Thread: " + number + " is an Armstrong number");
            } else {
                System.out.println("Armstrong Thread: " + number + " is not an Armstrong number");
            }

        } else if (threadName.equals("Reverse")) {
            int temp = number, reverse = 0;
            while (temp > 0) {
                int digit = temp % 10;
                reverse = reverse * 10 + digit;
                temp /= 10;
            }
            System.out.println("Reverse Thread: Reverse of " + number + " = " + reverse);
        }
    }
}
```

INPUT

```
153
```

OUTPUT

```
Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351
```

Q2. Multiple thread classes

Score: 20.00 **TC: 3/3 passed**

CODE

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        PrimeThread p = new PrimeThread(n);
        FactorialThread f = new FactorialThread(n);
        ReverseThread r = new ReverseThread(n);

        p.start();
        f.start();
        r.start();

        p.join();
        f.join();
        r.join();
    }
}

class PrimeThread extends Thread {
    int n;

    PrimeThread(int n) {
        this.n = n;
    }

    public void run() {
        synchronized (Main.class) {
            while (Order.turn != 1) {
                try {
                    Main.class.wait();
                } catch (Exception e) {
                }
            }

            boolean prime = true;

            if (n <= 1) {
                prime = false;
            } else {
                for (int i = 2; i <= Math.sqrt(n); i++) {
                    if (n % i == 0) {
                        prime = false;
                        break;
                    }
                }
            }

            if (prime)
                System.out.println("Prime: " + n + " is a prime number");
            else
                System.out.println("Prime: " + n + " is not a prime number");

            Order.turn = 2;
            Main.class.notifyAll();
        }
    }
}
```

```
class FactorialThread extends Thread {
    int n;

    FactorialThread(int n) {
        this.n = n;
    }

    public void run() {
        synchronized (Main.class) {
            while (Order.turn != 2) {
                try {
                    Main.class.wait();
                } catch (Exception e) {
                }
            }

            long fact = 1;

            for (int i = 1; i <= n; i++) {
                fact = fact * i;
            }

            System.out.println("Factorial: " + n + "! = " + fact);

            Order.turn = 3;
            Main.class.notifyAll();
        }
    }
}

class ReverseThread extends Thread {
    int n;

    ReverseThread(int n) {
        this.n = n;
    }

    public void run() {
        synchronized (Main.class) {
            while (Order.turn != 3) {
                try {
                    Main.class.wait();
                } catch (Exception e) {
                }
            }

            int temp = n;
            int reverse = 0;

            while (temp != 0) {
                reverse = reverse * 10 + temp % 10;
                temp = temp / 10;
            }

            System.out.println("Reverse: Reverse of " + n + " = " + reverse);

            Order.turn = 4;
            Main.class.notifyAll();
        }
    }
}

class Order {
```

```
    static int turn = 1;
}
```

INPUT

```
7
```

OUTPUT

```
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
```

Q3. Thread Synchronization

Score: 10.00 **TC: 3/3 passed**

CODE

```
class Main {
    public static void main(String[] args) throws Exception {
        Counter co = new Counter();

        ThreadA T1 = new ThreadA(co);
        ThreadB T2 = new ThreadB(co);

        T1.start();
        T2.start();

        T1.join();
        T2.join();

        System.out.println("Final Count = " + co.count);
    }
}

class Counter {
    int count = 0;

    public synchronized void counterIncrement() {
        count++;
    }
}

class ThreadA extends Thread {
    Counter c;

    ThreadA(Counter co) {
        c = co;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.counterIncrement();
        }
        System.out.println("ThreadA completed");
    }
}

class ThreadB extends Thread {
    Counter c;

    ThreadB(Counter co) {
        c = co;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.counterIncrement();
        }
        System.out.println("ThreadB completed");
    }
}
```

OUTPUT

```
ThreadA completed
ThreadB completed
Final Count = 2000
```